

Documentation of WRO Future Engineer 2023



By Team Sunflower 🌻

This document contains:

Abstract	1
Team members and the vehicle	2
Engineering information	4
Engineering Factor	10
Obstacle Management	15
Links and video	21

Abstract

About our team:

Our team, ***Sunflower***, contains three members who are all students at Yothinburana School. We have been dedicating our afterschool and weekend hours to developing and refining our robot, putting a lot of hard work, creativity, and passion into our project.

How we make our robot:

We used LEGO parts to create our robot's frame and attach its controller and other components. LEGO is an accessible building material that comes in various shapes, allowing us to create a flexible and customizable robot. We chose to use LEGO because of its versatility and ease of use.

Future Engineer this year:

Our school is participating in an international competition for the second time. We are determined to learn from the mistakes made by our seniors last year in Dortmund and during the national competition. We have made significant changes to our robot's components. For example, during the national competition, we used Arduino UNO as a controller. However, it only had one serial port, which forced us to disconnect the compass wire every time we uploaded the program. This made it vulnerable to breakage. We have since developed a new strategy to ensure its success in completing the assigned tasks.

1. Team members and the vehicle:



As representatives from Thailand, we were thrilled to be the first on our national stage. However, that was just the beginning of our journey to the international stage. Our goal is to rank in the top 5 of this tournament, to prove that Thailand is not lacking in robotics, and to showcase our capabilities to the world. The name **Sunflower** is the name of our senior mentor; his name is Tawan in Thai, or if translated, his name will be Sunflower. He attended the international last year at Dortmund but didn't succeed so we want to carry on his legacy.

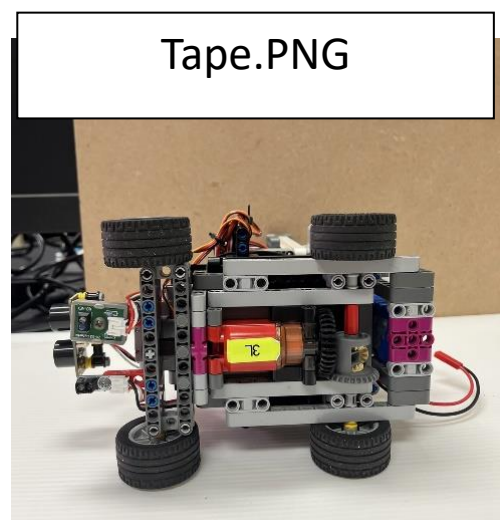
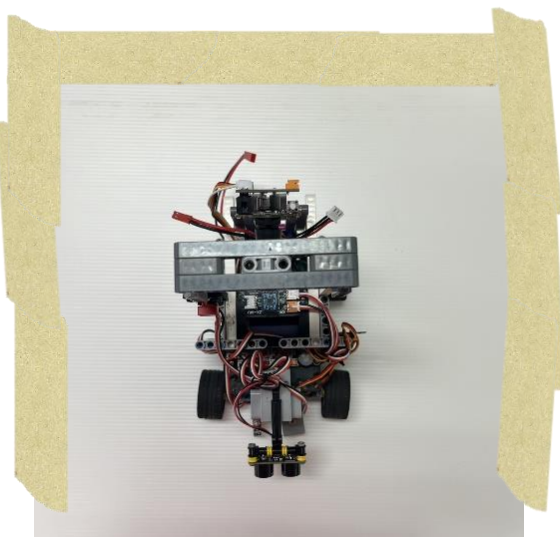
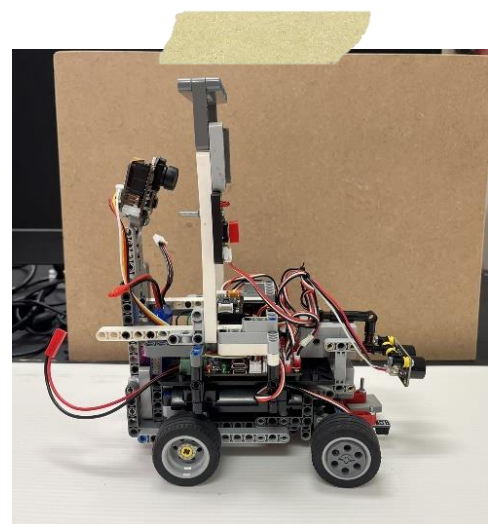
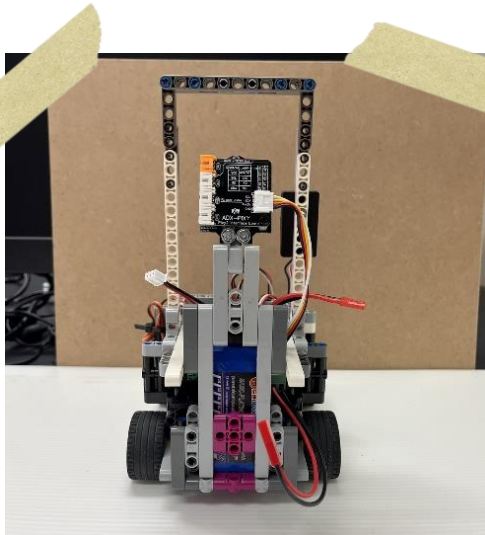
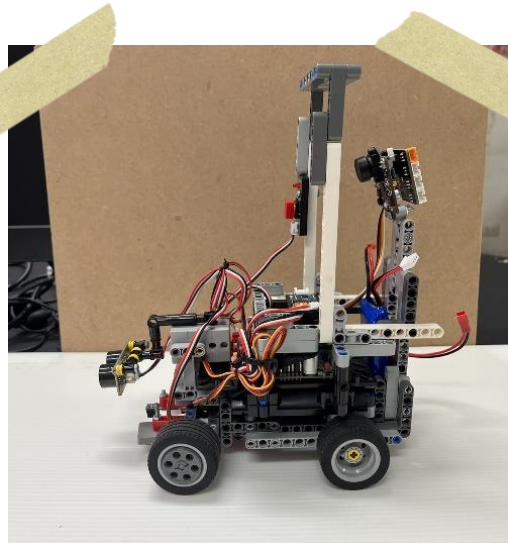
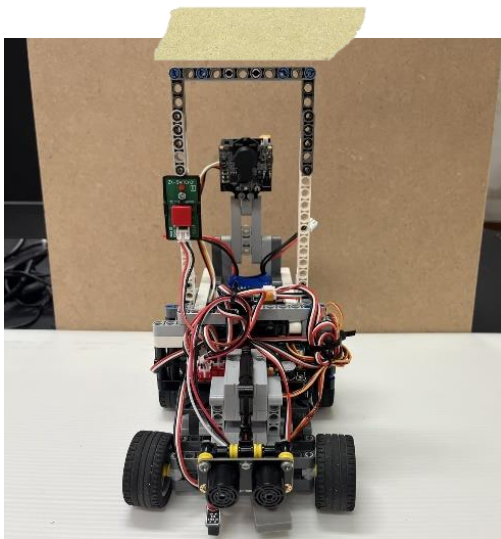
Our Team members and their roles:

The person on the left, dressed in a white shirt, is Mr. Vorawet Narkglom, who serves as the primary programmer for our team. Mr. Narkglom, a gifted Grade 10 student in the Yothinburana School English Program, is only 15 years of age. he plays a crucial role in bringing our robot to life.

The person in the middle wearing a red shirt is Mr. Vichaiwat Koonsap, our document designer who is writing this document 😊👍. I am 15 years old and currently studying in the regular program at Yothinburana.

The last person to introduce is Mr. Sakolnat Raktaengan, the designer of our robot. Sakolnat, who is a 15-year-old Grade 10 student in the Yothinburana School English Program, brings his creative and technical expertise to the physical design of our robot.

Vehicle Photos ⚙️ :

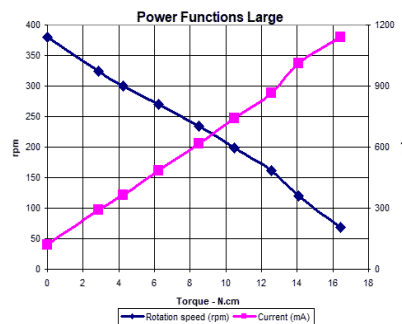


2. Engineering information

2.1 Mobility management

1. Power Function Large motor by Lego

We chose to use this particular motor because of its ability to move our robot effortlessly and with high speed. Having a motor with insufficient power output would have been a real problem. Another reason for our choice was that it is easy to modify. We can simply cut the tip of the wire and solder it with our wire of choice.



Technical Information

-Speed: 390rpm

-Maximum torque: 40N/cm

Voltage: 9v

2.GEEKSERVO 2kg 360 Degrees

It may look like a regular servo (because it is), but we specifically chose it because it can easily attach to our robot, which is mostly made out of Lego. As the name suggests, it can rotate 360 degrees, which is more than sufficient for our steering wheel and ultrasonic rotation needs.

Technical information

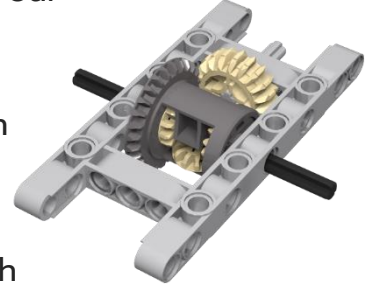
- Rotation: 360 Degree
- Start Voltage: 2.5V
- Working Voltage: 3.3-6.0V
- Rated Voltage: 4.8V
- Rated Current: 70mA
- Maximum torque: 1.6 0.2 kg-cm (4.8V)
- Angle rotation speed: 60 / 0.14s



3. Technic, gear differential with inner tabs and closed center, 28 bevel teeth

This gearbox is an essential component in our robot's mechanics. It's a Lego Technic gearbox designed to transmit the motor's power to our robot's wheels effectively. Here's how it works:

1. We start by attaching a gear to the motor shaft. When the motor operates, this gear begins to rotate.
2. Next, we insert the gearbox, which features a matching gear on the front side. This gear meshes with the one we placed on the motor, creating a connection.
3. To drive the robot's wheels on both sides, we utilize a smaller-sized gear. This smaller gear engages with the gears inside the gearbox, ensuring that the wheels receive the rotational power.



This gearbox mechanism ensures efficient power transmission from the motor to the wheels, enabling our robot to move effectively and precisely.

4. Lego 49.5 * 20 wheel

After testing all the wheels we could find or have, we found this one. Why do we need to choose this? Well, it's because it has the perfect size and shape to fit into our robot. The perfect size means that if the wheel is too big, it will be too fast, or the motor doesn't have enough power to make the robot walk, or if it's too small, the speed will drop due to the small rotation of the wheel.



2.2 Power and Sense Management

1. Pixy version 2.1

The camera plays a crucial role in the final round of the race as it needs to detect the objects on the track, whether they are red or green. We use this camera because of its fast detection speed of 60 fps, which is faster when compared to Huskylens. However, the camera has its pros and cons. The good thing is that it is fast, but the downside is that it cannot detect objects clearly if they are too dark. This was the reason why our seniors lost last year. Fortunately, we were able to fix the problem.

Software and Library: <https://pixycam.com/downloads-pixy2/>



Technical information

- Resolution: 1296 x 976
- Processor: NXP LPC4330
- Clock speed: 204 MHz
- Ram: 264KB
- Flash: 2 MB
- Framerate: 60 fps
- Connection: USB, UART, SPI, I²C

2. Gravity URM 09

The ultrasonic sensor plays a vital role in guiding the robot's navigation by accurately detecting distances between the robot and potential obstacles. Its precision in measuring distances ensures that the robot can maneuver and make informed decisions to navigate its path safely and effectively, avoiding collisions and maintaining its course.

Technical information

- Supply Voltage: 3.3~5.5V DC
- Operating Current: 20mA
- Operating Temperature Range: -10°C~+70°C
- Measurement Range: 2cm~500cm (can be set)
- Resolution: 1cm
- Frequency: 50Hz Max

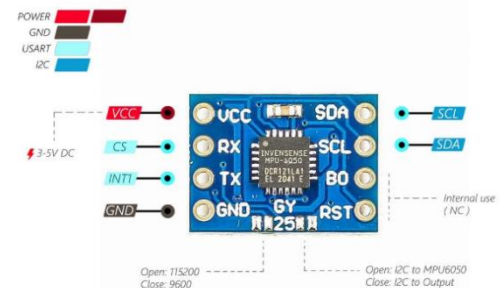


3. Gy-25

A gyro is a component that enables a robot to determine its orientation and turn in the appropriate direction. We chose this gyro sensor specifically because of how effective it is. It also comes in a very small size to attach to our robot.

Technical information

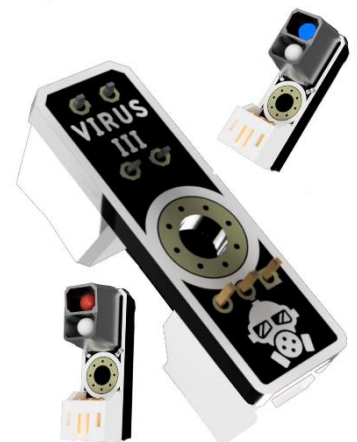
- Chip: MPU-6050
- Power voltage: 3 - 5V
- Communication mode: Serial communication (baud 9600,115200), IIC communication
- Model Size: 15.5mm*11.5mm
- Pitch: 2.54mm
- Direct Data: YAW ROLL PITCH
- Heading angle (YAW): $\pm 180^\circ$
- Roll angle (ROLL): $\pm 180^\circ$
- Pitch angle (PITCH): $\pm 180^\circ$



GY-25 Pinout

4. Virus III

The color sensors play an important role in both rounds, as we use them for line detection. There are 2 lines with different colors in the corner of the race field, which is why we use 2 different colors of the color sensor, blue and red. The blue color sensor is used for detecting both colors, while the red color sensor is only used for the blue line because it can't see the red line. With these color sensors, the robot will be able to detect the line and check if it is red or blue. If it is red, the robot will turn right, or if it's blue, it will turn left. The Virus III sensor comes with a very fast respond time and very accurate performance.



6. ZX-Switch 01

This switch gives us an easier way to start the robot. Since the controller board came with switches like OK, Switch A, Switch B, and ON/OFF, it is hard to push, so we found this switch that could be attached to the frame outside the board.



5. POP-32i

The controller plays the most important role in our robot, the brain. The controller is used to connect the wires from all components and download the code to make it run. It's a 32-bit controller with the STM32F103CBT6 microcontroller. To download the code into our robot, we use the Type-C wire to connect to the port on the board and download it with the Arduino 2.2.1 software and POP32 library. We started using the Arduino Uno controller board on the national stage, but then we realized that it was difficult to use. Then, after we won the national stage, we started researching the controller board. We came up with a lot of ideas, such as the Raspberry Pi or even making our own board, but since it's too expensive and hard to use, we researched the company that makes the custom board, and we found that this POP-32 board is popular in Thailand, so we bought one and tested it, we found it to provide all the necessary features and capabilities required for our robot.

Technical information

-Clock Signal: The system operates with a 20MHz clock signal from a ceramic resonator.

-JST 3-Pin Ports: It provides JST 3-pin connectors with 11 ports for connecting sensors and peripheral devices.

-LED Indicators: Includes LED indicators for power status, low battery alert, and USB port connection status.

-RESET Switch: Features a RESET switch for resetting the microcontroller.



- USB Port: Provides a USB port for program downloading and data communication with a computer.
- Power Input: Has a power input port via a screw terminal block that accepts power from 6V to 9V with an on/off switch for power control.
- Battery Compatibility: Designed for use with 2-cell lithium polymer batteries (7.4V to 8.4V).
- Voltage Regulation: Includes voltage regulation to supply 3.3V to the microcontroller, OLED display, and main I/O ports.
- Digital and Analog Ports: Offers 9 digital or analog input/output ports (A0 to A8) for external signal connections.
- I2C Ports: Provides 2 sets of digital ports for I2C communication (SDA and SCL) using PH4 connectors in GROVE format.
- UART Port: Includes a UART port for data communication, with RXD and TXD pins on PB7 and PB6.
- Motor Driver Circuit: Features a 4-channel motor driver circuit with LED status indicators, supporting motors from 3V to 12V, with a current rating of 1.5A per channel and a maximum of 2A per channel, along with overcurrent protection.
- Servo Motor Ports: Offers 6 servo motor ports for driving servo motors.
- Speaker: Includes a piezo speaker for generating sound, connected to PB5.
- OLED Display: Features a 1.5-inch OLED display with a resolution of 128 x 64 pixels, capable of displaying graphics and text, and communicates via I2C.
- Push Buttons and Knob: Includes push buttons (OK, SW-A, SW-B) and an adjustable knob (KNOB) for input and testing.

6. HeliCoxNano 7.4V 1100 mAh 30C Li-Po battery

The battery provides the power for our robot. This battery is very lightweight and compact to fit into the robot since the robot is very small and compact. And it is also able to connect to our controller board. We calculated the amount of mAh that is enough to power our robot for 1-2 hours and then recharged.

Technical information

- 2 cells Voltage: 7.4V
- Capacity: 1100mAh 30C
- Can be charged at a current of up to 5 times the capacity (5C)
- Connectors are in a JST type that can be easily disconnected.

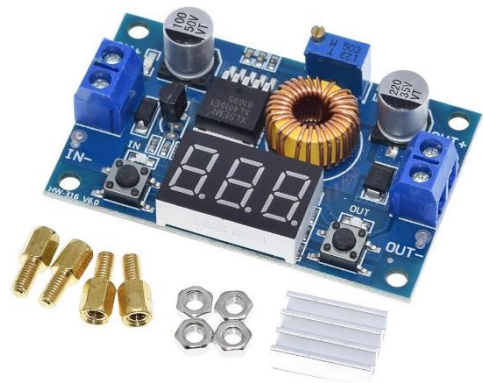


6. HW-316 V6.0

This is a step-down DC-DC module. It comes with a status indicator light, a display screen that shows the voltage meter, and self-calibration of the voltage meter. The electrical voltage has an error of 0.05 V, with a measuring range of 0–40 V. We need this step down to show us how long until we need to recharge the battery. Before this, we need to check every 30 minutes. Now, we can use it without worrying about when to recharge.

Technical information

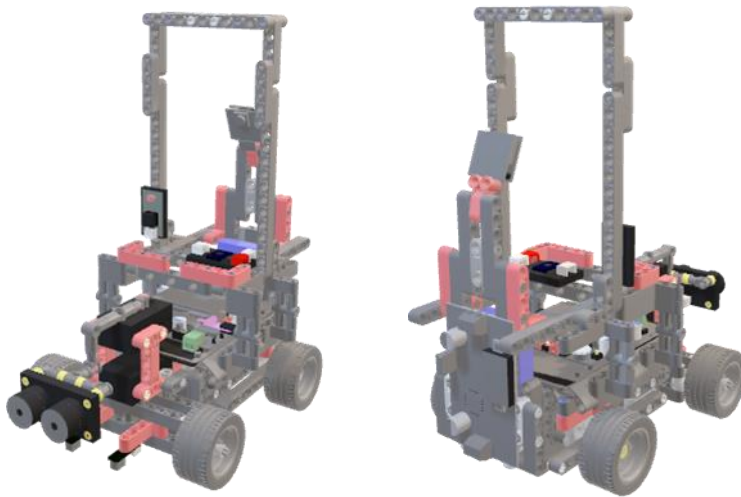
- Input voltage: DC 4.0 ~ 38V
- Output voltage: DC 1.25V ~ 36V continuously adjustable
- Output current: max 5A,
- Output power: up to 75W
- Voltmeter error: $\pm 0.05V$
- Measure range: 0 ~ 40V
- Conversion efficiency: up to 96%
- Load regulation: $S(I) \leq 0.8\%$
- Voltage Regulation: $S(u) \leq 0.8\%$



3. Engineering Factor

We designed this robot using Lego parts and bolts. We had a model from our senior last year last year, but we made an upgrade to make it better. We researched the components that give us the specification we want. And then we started ordering components, but our school can offer us the Lego parts. Then we started building it. We changed the robot around three times before we used this one. But we still have to adjust some parts, like the steering angle or camera angle.

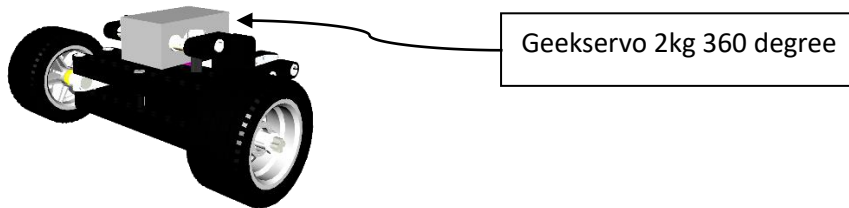
Our robot 3D model



Part explanation

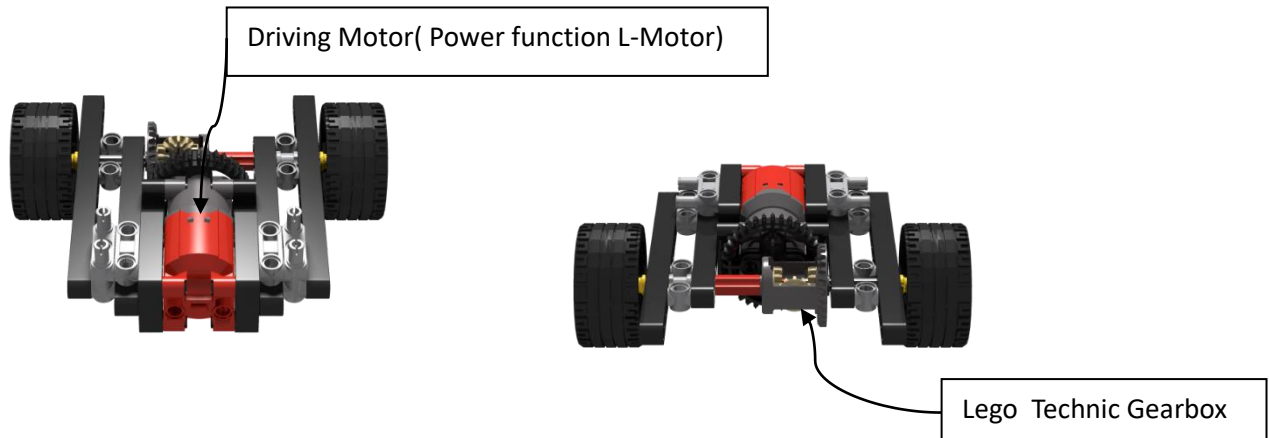
(All the parts photo are rendered from 3D model we draw ourselves)

1. Steering part



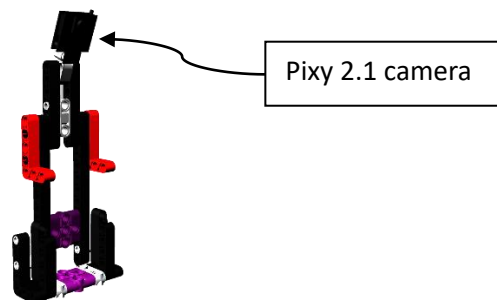
We use Lego parts to attach the steering servo, which has a Lego design to put connector pegs inside and then link it to the steering part. This part is used for turning left or right.

2. Driving part



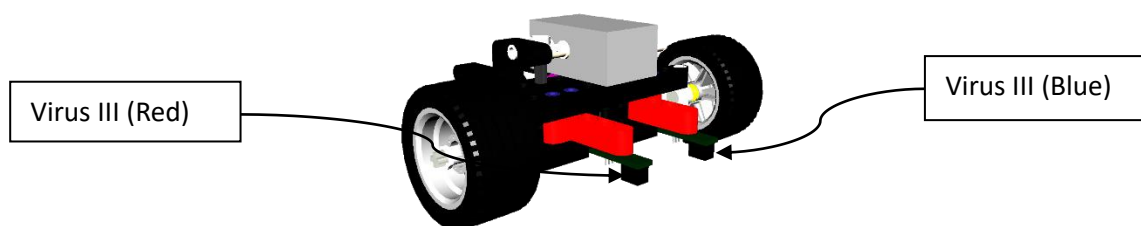
The power Function Large motor is already a Lego part so all we need is where to place it. We placed it underneath the robot and use the Lego gearbox to drive on 2 side. We used this part for moving the robot forward

3. Camera



The camera needs the right angle so we set it at the back of the robot and high. We used bolts to link the Pixy to the frame. But the problem is the wire, some wire color is red which Pixy can detect it as a red obstacle block so we need to place it somewhere that it can't see the wire and put the wire away. This part plays the main role in the round with obstacle. It detected what color the block is determine which way to turn

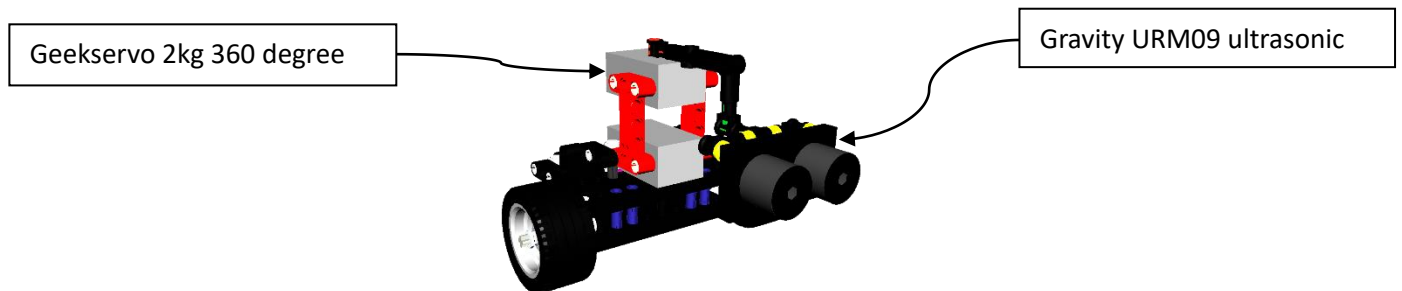
4. Color sensor



This part is simple. We link it to the steering part by using 2 Lego angular beam 2x4 and use a bolt to join it with sensor. This part will detect the color line on the field which is red and blue. It will know when and which way to turn. It will also help to stop the robot. The sensor will check for 12

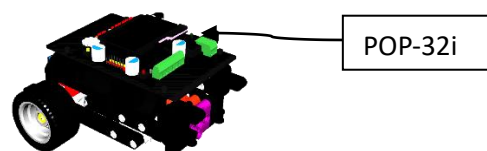
line with single color example if it detect the first line as red it will continue counting red for 12 lines and then will walk with timer until the time we set in the program ran out.

5. Ultrasonic sensor



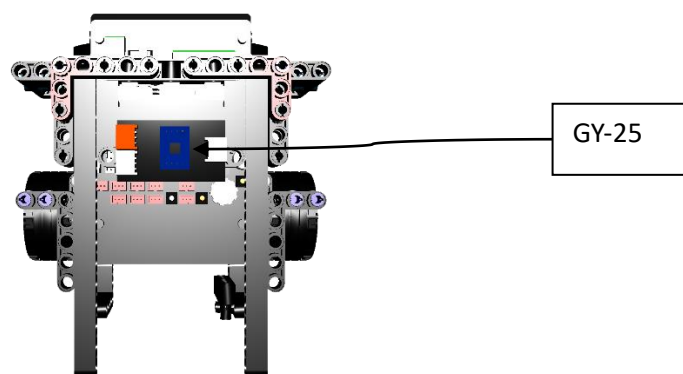
We use the same type of servo as the steering part. But this time, we place it on top of it and using connector peg. Then we link it to ultrasonic using Lego parts and bolt. Ultrasonic sensor will check the distance between the wall and the robot. It will maintain the safe distance so that the robot won't crash into the wall

6. Controller



This part, we use a double-sided tape and bolt. When we bought the board, it came with the protective pad. We use bolt to join the board and its pad. Then we use double-sided tape to place it on top of the Lego part

7. Gyro sensor



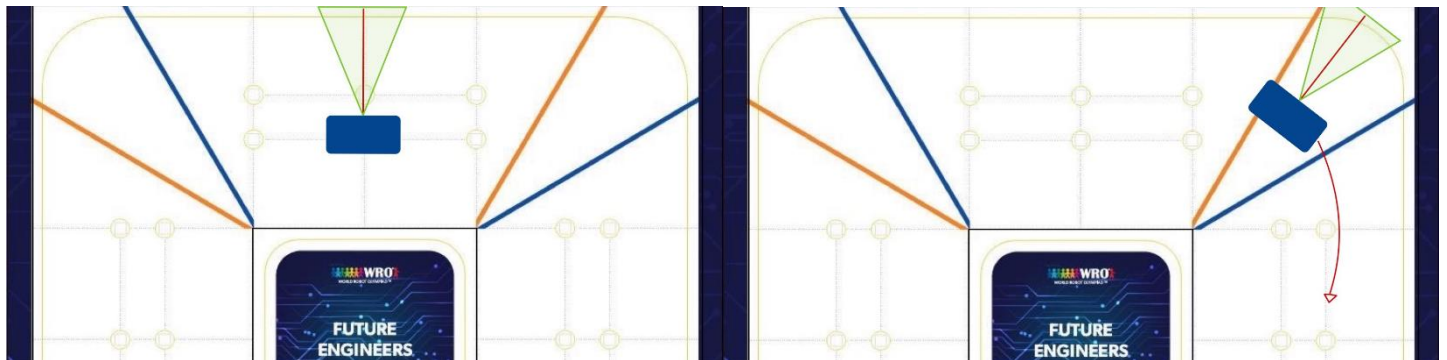
The gyro we ordered came with a case. The case has a hole that used for bolt. We use the bolt to attach it to the robot on top of the board

4. Obstacle Management

4.1 Qualification Round

In the qualification round, the robot will use the ultrasonic to measure the distance of the robot from the outer wall. The robot will use the color sensor to check the line color. If the first color is red, the robot will turn right, and if it's blue, the robot will turn left. Every time, the robot will check how many times it has crossed the first line. It will count 12 lines, and after that, it will walk with the timer that we set until the time we set runs out.

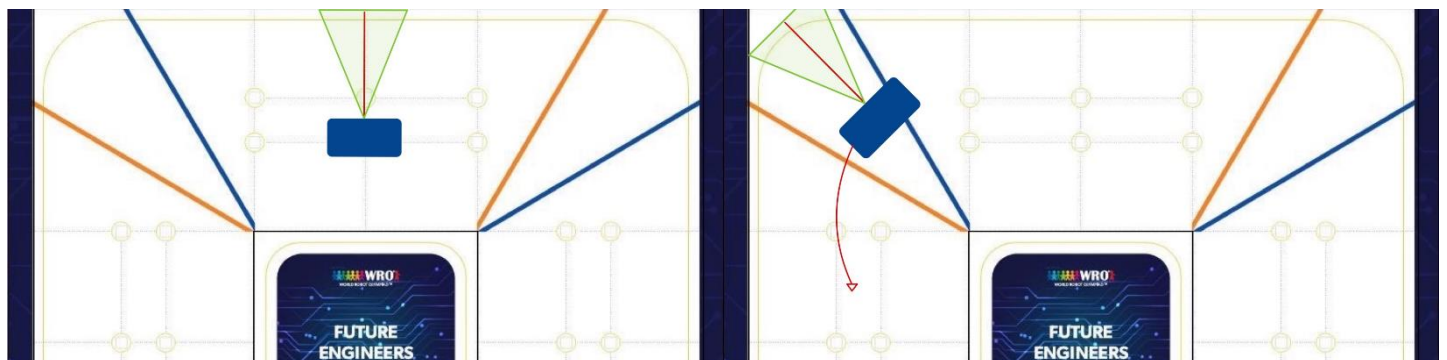
If the first line is red



Start

When detect the first line

If the first line is blue



Start

When detect the first line

The robot will use the distance of robot from the wall and gyro sensor degree to calculate into steering degree (turning degree) to maintain the distance between the wall with PID (Proportional Integral Derivative)

$$U(t) = k_p e(t) + k_i \int e(t) dt + k_d \frac{de}{dt}$$

(PID Formular)

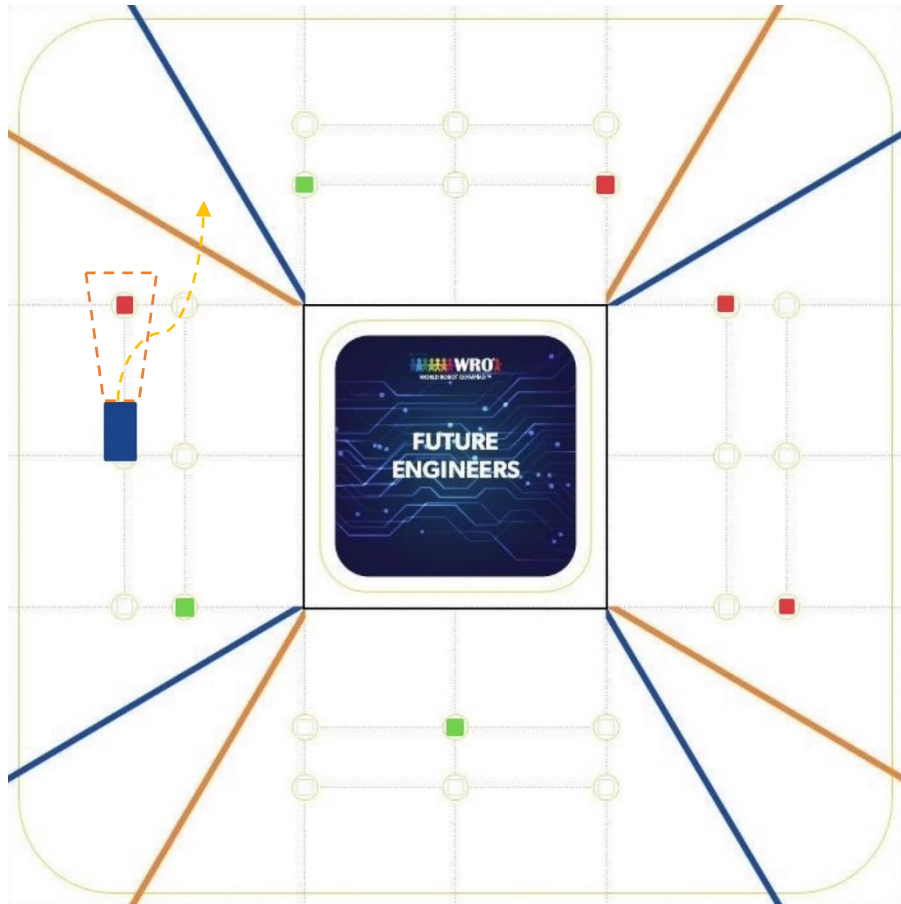
Our robot qualification round code

```
motor(4, 20);
delay(400);
motor(4, 100);
while (analogRead(BUTTON) > 500) {
    getTaco();
    line_detection();
    int wall_distance = getDistance();
    motor_and_steer((1 * x) * compassPID.Run((x * pvYaw) + ((wall_distance - 15.5) * 1.2) * ((float(TURN == 'TURN') - 0.5) * 2)));
    ultra_servo(-pvYaw, TURN);
    if (count >= 12) {
        long timer01 = millis();
        while (millis() - timer01 < 1000) {
            getTaco();
            line_detection();
            motor_and_steer((1 * x) * compassPID.Run((x * pvYaw) + ((wall_distance - 15.5) * 1.2) * ((float(TURN == 'TURN') - 0.5) * 2)));
            ultra_servo(-pvYaw, TURN);
        }
        motor(4, 0);
        while (true) {
        }
    }
    // motor_and_steer(-1 * compassPID.Run(-pvYaw + ((wall_distance - 25) * 1) * ((float(TURN == 'R') - 0.5) * 2)));
    // ultra_servo(-pvYaw, 'TURN');
}
motor(4, 0);
while (analogRead(BUTTON) <= 500)
;
while (analogRead(BUTTON) > 500)
;
while (analogRead(BUTTON) <= 500)
;
;
```

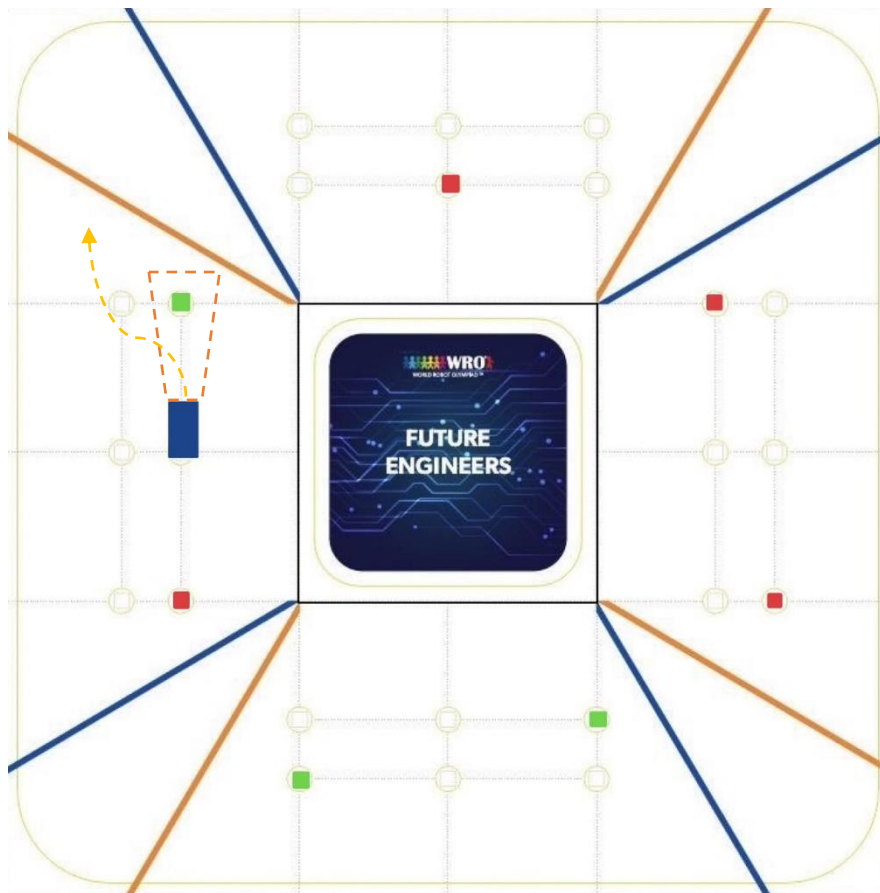
4.2 Final Round

In this round, the robot still use ultrasonic, color sensor, and gyro. But, we added the use of Pixy. We used Pixy camera to check if there are any obstacles, if yes then check what color it is. If it's red the robot will turn right or if it's green the robot will turn left. It will use all the sensors to calculate steering degree to avoid the obstacle and maintaining distance from the wall.

If the robot sees red obstacle.



If the robot sees green obstacle



The robot still use the PID to walk, but we added the avoidance degree to avoid the obstacles

Our obstacle avoidance code

```
float calculate_avoidance() {
    int blocks = pixy.ccc.getBlocks();

    found_block = false;

    if (blocks) {
        int signature = -1;
        int targetHeight = 10;
        float focalLength = 2.3;
        float cameraFOV = 80.0;

        int largestBlockIndex = -1;
        int largestBlockArea = 0;

        for (int i = 0; i < blocks; i++) {
            if (pixy.ccc.blocks[i].m_height > 1.33 * float(pixy.ccc.blocks[i].m_width)) {
                int objectArea = pixy.ccc.blocks[i].m_width;
                found_block = true;
                if (objectArea > largestBlockArea) {
                    largestBlockIndex = i;
                    largestBlockArea = objectArea;
                    signature = pixy.ccc.blocks[i].m_signature;
                }
            }
        }

        if (signature != -1) {
            int objectHeight = pixy.ccc.blocks[largestBlockIndex].m_height;
            float distance = (targetHeight * focalLength * 100) / objectHeight;

            float blockCenterX = pixy.ccc.blocks[largestBlockIndex].m_x;
            float blockCenterY = pixy.ccc.blocks[largestBlockIndex].m_y;

            float deltaX = blockCenterX - pixy.frameWidth / 2;
            float deltaY = blockCenterY - pixy.frameHeight / 2;

            float detected_degree = deltaX * 40 / pixy.frameWidth;

            float blockPositionX = distance * sin(degreesToRadians(detected_degree));
            float blockPositionY = distance * cos(degreesToRadians(detected_degree)) - 16;

            if (signature == 1) {
                avoidance_degree = max(radiansToDegree(atan2(blockPositionX + 10.7, blockPositionY)), 6.7);
                Blocks_TURN = 'TURN';
                last_block = 'R';
                before_last_block = 'R';
            } else {
                avoidance_degree = min(radiansToDegree(atan2(blockPositionX - 8.5, blockPositionY)), -4.5);
                last_block = 'L';
                Blocks_TURN = 'TURN';
                before_last_block = 'L';
            }
        }

        // Serial.print("Detected degree: ");
        // Serial.print(detected_degree);
        // Serial.println(" degree ");
        // Serial.print("Position (X, Y): ");
        // Serial.print(blockPositionX);
        // Serial.print(", ");
        // Serial.print(blockPositionY);
        // Serial.print(" cm   Degree rotate: ");
        // Serial.println(avoidance_degree);
    }
}

return avoidance_degree;
}
```

5. Links and video

5.1 Github

Github Link- https://github.com/Snackels/WRO_Future-Engineer_Sunflower

5.1 YouTube

Qualification round: <https://youtu.be/PkhhVm6EGIA?si=AeqNGVa3Cn8eCuS9>

Final round: https://youtu.be/4hqVN3C_IVI

Documentary video: https://youtu.be/v2E_7PV4V1o